

# CA1: Dataframe Manipulation with Spotify Data

## Introduction

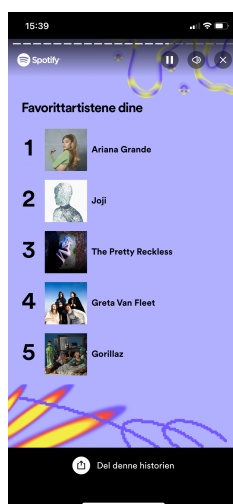
Pandas is an extremely powerful tool to handle large amounts of tabular data. In this compulsory assignment, you will use Pandas to explore one of the TA's personal spotify data in depth.

Additional information:

- Feel free to create additional code cells if you feel that one cell per subtask is not sufficient.
- Remember, Pandas uses very efficient code to handle large amounts of data. For-loops are not efficient. If you ever have to use a for-loop to loop over the rows in the DataFrame, you have *probably* done something wrong.
- Label all graphs and charts if applicable.

## Task

I typically enjoy indie and rock music. I am a big fan of everything from old-fashioned rock and roll like Led Zeppelin and Jimi Hendrix, to newer indie artists like Joji and Lana Del Rey. This is why my spotify wrapped for 2023 came as quite a surprise:



Now, I'm no hater of pop music, but this was unexpected. For this assignment, you will investigate my listening habits, including a deep dive into my Ariana Grande listening habits, and try to find an answer to why she was my top artist; was there a fault in the spotify algorithm? Am I actually secretly an *Arianator*? (yes, I did have to look that up).

Or am I just lying to myself about how often I listen to guilty pleasure music?

## Part 1: Initial loading and exploration

### 1.0 Import necessary libraries:

pandas, numpy, matplotlib.pyplot (other libraries such as seaborn or plotly are also allowed if you want prettier plots). It might also be a good idea to use **os** for task 2.0

```
In [2]: import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
import seaborn as sns
import glob
# ---- Insert other imports ----
```

### 1.1 Loading the data

Load the dataset in the file `streaming_history_0.csv` into a Pandas DataFrame called `df_spotify_0`.

```
In [3]: df_spotify_0 = pd.read_csv('spotify_data/streaminghistory0.csv')
```

### 1.2 Help function

Use the Python command `help` to help you understand how to use the `pd.DataFrame.head` and `pd.DataFrame.tail` methods.

```
In [4]: help(pd.DataFrame.head)
help(pd.DataFrame.tail)
```

Help on function head in module pandas.core.generic:

```
head(self, n: 'int' = 5) -> 'Self'  
    Return the first `n` rows.
```

This function returns the first `n` rows for the object based on position. It is useful for quickly testing if your object has the right type of data in it.

For negative values of `n`, this function returns all rows except the last `|n|` rows, equivalent to ``df[:n]``.

If n is larger than the number of rows, this function returns all rows.

Parameters

-----

n : int, default 5  
 Number of rows to select.

Returns

-----

same type as caller  
 The first `n` rows of the caller object.

See Also

-----

DataFrame.tail: Returns the last `n` rows.

Examples

-----

```
>>> df = pd.DataFrame({'animal': ['alligator', 'bee', 'falcon', 'lion',  
...                               'monkey', 'parrot', 'shark', 'whale', 'zebra']})  
>>> df
```

```
   animal  
0  alligator  
1      bee  
2   falcon  
3     lion  
4   monkey  
5   parrot  
6    shark  
7    whale  
8     zebra
```

Viewing the first 5 lines

```
>>> df.head()  
   animal  
0  alligator  
1      bee  
2   falcon  
3     lion  
4   monkey
```

Viewing the first `n` lines (three in this case)

```
>>> df.head(3)
      animal
0  alligator
1        bee
2      falcon
```

For negative values of `n`

```
>>> df.head(-3)
      animal
0  alligator
1        bee
2      falcon
3        lion
4      monkey
5      parrot
```

Help on function tail in module pandas.core.generic:

```
tail(self, n: 'int' = 5) -> 'Self'
    Return the last `n` rows.
```

This function returns last `n` rows from the object based on position. It is useful for quickly verifying data, for example, after sorting or appending rows.

For negative values of `n`, this function returns all rows except the first `|n|` rows, equivalent to ``df[|n|:]``.

If n is larger than the number of rows, this function returns all rows.

Parameters

-----

n : int, default 5  
 Number of rows to select.

Returns

-----

type of caller  
 The last `n` rows of the caller object.

See Also

-----

DataFrame.head : The first `n` rows of the caller object.

Examples

-----

```
>>> df = pd.DataFrame({'animal': ['alligator', 'bee', 'falcon', 'lion',
...                               'monkey', 'parrot', 'shark', 'whale', 'zebra']})
>>> df
      animal
0  alligator
1        bee
2      falcon
3        lion
4      monkey
```

```
5    parrot
6    shark
7    whale
8    zebra
```

Viewing the last 5 lines

```
>>> df.tail()
      animal
4  monkey
5  parrot
6   shark
7   whale
8   zebra
```

Viewing the last `n` lines (three in this case)

```
>>> df.tail(3)
      animal
6  shark
7  whale
8  zebra
```

For negative values of `n`

```
>>> df.tail(-3)
      animal
3    lion
4  monkey
5  parrot
6   shark
7   whale
8   zebra
```

### 1.3 Getting an overview

Print the first five and last ten rows of the dataframe. Have a quick look at which columns are in the dataset.

```
In [5]: print('first 5 rows')
print(df_spotify_0.head(5))
print('last 10 rows')
print(df_spotify_0.tail(10))
```

```

first 5 rows
      endTime      artistName      trackName \
0  2022-12-03 02:02  Cigarettes After Sex      Truly
1  2022-12-03 02:02      Leonard Cohen  Take This Waltz - Paris Version
2  2022-12-06 21:05      Vlad Holiday      So Damn Into You
3  2022-12-06 21:05      Lorde      Team
4  2022-12-06 21:05      Ariana Grande      Into You

      msPlayed
0      30000.0
1       8210.0
2      37895.0
3       8984.0
4       1221.0
last 10 rows
      endTime      artistName      trackName \
11949  2023-01-02 20:58      Ariana Grande      six thirty
11950  2023-01-02 20:58      Leonard Cohen      Thanks for the Dance
11951  2023-01-02 20:59      Des Rocs      Used to the Darkness
11952  2023-01-02 20:59  Caroline Polachek      Hit Me Where It Hurts
11953  2023-01-02 20:59  Caroline Polachek      Hit Me Where It Hurts
11954  2023-01-02 20:59  Kaizers Orchestra      Resistansen
11955  2023-01-02 20:59      Mr.Kitty      After Dark
11956  2023-01-02 20:59      daddy's girl  after dark x sweater weather
11957  2023-01-02 20:59      daddy's girl  after dark x sweater weather
11958  2023-01-02 20:59      daddy's girl  after dark x sweater weather

      msPlayed
11949      1699.0
11950     19483.0
11951       185.0
11952       603.0
11953       208.0
11954       208.0
11955    101447.0
11956       301.0
11957       208.0
11958       789.0

```

## 1.4 Formatting correctly

When working with Pandas, it's very useful to have columns which contains dates in a specific format called *datetime*. This allows for efficient manipulation and analysis of time-series data, such as sorting, filtering by date or time, and resampling for different time periods. Figure out which column(s) would be appropriate to convert to datetime, if any, and if so, perform the conversion to the correct format.

```

In [6]: df_spotify_0['endTime'] = pd.to_datetime(df_spotify_0['endTime'])

print(df_spotify_0.dtypes)

```

```
endTime      datetime64[ns]
artistName    object
trackName     object
msPlayed      float64
dtype: object
```

In [ ]:

## 1.5 Unique artists

Find how many unique artists are in the dataset.

```
In [7]: unique_artists = df_spotify_0['artistName'].nunique()

print('Number of unique artists: ', unique_artists)
```

Number of unique artists: 495

## 1.6 Unique songs

Find how many unique songs are in the dataset.

```
In [8]: unique_songs = df_spotify_0['trackName'].nunique()

print('Number of unique songs: ', unique_songs)
```

Number of unique songs: 1308

## Part 1: Questions

Q1: Which columns are in the dataset?

A1: endTime, artistName, trackName, msPlayed are the columns in the dataset.

Q2: What timeframe does the dataset span?

A2: from: 2022-12-03 02:02 to: 2023-01-02 20:59 which is about a month.

Q3: How many unique artists are in the dataset?

A3: There are 495 unique artists in the dataset.

Q4: How many unique songs are in the dataset?

A4: There are 1308 unique songs in the dataset.

## Part 2: Working with all the data

### 2.0 Importing all the dataframes

In Task 1, you only worked with about a month worth of data. Now, you will work with over a year worth.

In the *spotify\_data* folder, there is more than just one listening record. Load each of the 14 listening records into a dataframe (1 dataframe per listening record), and concatenate them together into one large dataframe named `df`.

```
In [9]: files = glob.glob('spotify_data/*.csv')
dfs = [pd.read_csv(file) for file in files]

df = pd.concat(dfs, ignore_index=True)
df.head()
```

```
Out[9]:
```

|   | endTime             | artistName             | trackName                                 | msPlayed |
|---|---------------------|------------------------|-------------------------------------------|----------|
| 0 | 2023-03-16<br>10:36 | Mr.Kitty               | After Dark                                | 998.0    |
| 1 | 2023-03-16<br>10:36 | CORPSE                 | MISA MISA!                                | 998.0    |
| 2 | 2023-03-16<br>10:36 | The Handsome<br>Family | Far from Any Road                         | 650.0    |
| 3 | 2023-03-16<br>10:36 | Mr.Kitty               | After Dark                                | 882.0    |
| 4 | 2023-03-16<br>10:36 | Led Zeppelin           | Over the Hills and Far Away -<br>Remaster | 185.0    |

## 2.1 Sorting by time

Datasets often aren't perfect. One example of an issue that could occur is that the time-based data might not be in chronological order. If this were to happen, the rows in your dataframe could be in the wrong order. To ensure this isn't an issue in your dataframe, you should sort the dataframe in chronological order, from oldest to newest.

```
In [10]: df['endTime'] = pd.to_datetime(df['endTime'])
sort_df = df.sort_values(by='endTime', ascending=True)
print(sort_df)
```



|        |            | endTime  | artistName           | \ |
|--------|------------|----------|----------------------|---|
| 83713  | 2022-12-03 | 02:02:00 | Cigarettes After Sex |   |
| 83714  | 2022-12-03 | 02:02:00 | Leonard Cohen        |   |
| 83715  | 2022-12-06 | 21:05:00 | Vlad Holiday         |   |
| 83716  | 2022-12-06 | 21:05:00 | Lorde                |   |
| 83717  | 2022-12-06 | 21:05:00 | Ariana Grande        |   |
| ...    |            | ...      |                      |   |
| 155476 | 2023-12-07 | 21:13:00 | Ariana Grande        |   |
| 155477 | 2023-12-07 | 21:14:00 | Ariana Grande        |   |
| 155478 | 2023-12-07 | 21:14:00 | Leonard Cohen        |   |
| 155479 | 2023-12-07 | 21:17:00 | The Vaccines         |   |
| 152570 |            | NaT      | The Lumineers        |   |

|        |                                 | trackName | msPlayed |
|--------|---------------------------------|-----------|----------|
| 83713  |                                 | Truly     | 30000.0  |
| 83714  | Take This Waltz - Paris Version |           | 8210.0   |
| 83715  | So Damn Into You                |           | 37895.0  |
| 83716  | Team                            |           | 8984.0   |
| 83717  | Into You                        |           | 1221.0   |
| ...    |                                 | ...       | ...      |
| 155476 | off the table (with The Weeknd) |           | 13448.0  |
| 155477 | my hair                         |           | 23757.0  |
| 155478 | Thanks for the Dance            |           | 9317.0   |
| 155479 | Your Love Is My Favourite Band  |           | 14661.0  |
| 152570 | Ophelia                         |           | 371.0    |

[167439 rows x 4 columns]

## 2.2 Setting a timeframe

For this investigation, we are only interested in investigating listening patterns from **2023**. Remove any data not from **2023** from the DataFrame.

```
In [11]: sort_df['endTime'] = pd.to_datetime(sort_df['endTime'])
df = sort_df[sort_df['endTime'].dt.year == 2023]
print(df)
```

|        |            | endTime  | artistName     | trackName                       |
|--------|------------|----------|----------------|---------------------------------|
| \      |            |          |                |                                 |
| 94594  | 2023-01-01 | 01:17:00 | Ariana Grande  | 7 rings                         |
| 94595  | 2023-01-01 | 01:17:00 | Ariana Grande  | 7 rings                         |
| 94596  | 2023-01-01 | 01:17:00 | Ariana Grande  | positions                       |
| 94597  | 2023-01-01 | 01:17:00 | Peach Pit      | Being so Normal                 |
| 94598  | 2023-01-01 | 01:17:00 | Kelly Clarkson | Santa, Can't You Hear Me        |
| ...    |            | ...      | ...            | ...                             |
| 155475 | 2023-12-07 | 21:13:00 | Lana Del Rey   | Art Deco                        |
| 155476 | 2023-12-07 | 21:13:00 | Ariana Grande  | off the table (with The Weeknd) |
| 155477 | 2023-12-07 | 21:14:00 | Ariana Grande  | my hair                         |
| 155478 | 2023-12-07 | 21:14:00 | Leonard Cohen  | Thanks for the Dance            |
| 155479 | 2023-12-07 | 21:17:00 | The Vaccines   | Your Love Is My Favourite Band  |

|        | msPlayed |
|--------|----------|
| 94594  | 139.0    |
| 94595  | 487.0    |
| 94596  | 417.0    |
| 94597  | 2205.0   |
| 94598  | 278.0    |
| ...    | ...      |
| 155475 | 38298.0  |
| 155476 | 13448.0  |
| 155477 | 23757.0  |
| 155478 | 9317.0   |
| 155479 | 14661.0  |

[156557 rows x 4 columns]

## 2.3 Deleting rows

Often in Data Science, you will encounter when a row entry has the value *NaN*, indicating missing data. These entries can skew your analysis, leading to inaccurate conclusions. For this task, identify and remove any rows in your DataFrame that contain NaN values.

Later in the course, you might encounter other techniques of dealing with missing data, typically referred to as *data imputation*. Here, though, you are just supposed to delete the entire rows with missing data.

```
In [12]: df = df.dropna()
         print(df)
```

|        |                     | endTime        | artistName                      | trackName |
|--------|---------------------|----------------|---------------------------------|-----------|
| \      |                     |                |                                 |           |
| 94594  | 2023-01-01 01:17:00 | Ariana Grande  | 7 rings                         |           |
| 94595  | 2023-01-01 01:17:00 | Ariana Grande  | 7 rings                         |           |
| 94596  | 2023-01-01 01:17:00 | Ariana Grande  | positions                       |           |
| 94597  | 2023-01-01 01:17:00 | Peach Pit      | Being so Normal                 |           |
| 94598  | 2023-01-01 01:17:00 | Kelly Clarkson | Santa, Can't You Hear Me        |           |
| ...    | ...                 | ...            | ...                             |           |
| 155475 | 2023-12-07 21:13:00 | Lana Del Rey   | Art Deco                        |           |
| 155476 | 2023-12-07 21:13:00 | Ariana Grande  | off the table (with The Weeknd) |           |
| 155477 | 2023-12-07 21:14:00 | Ariana Grande  | my hair                         |           |
| 155478 | 2023-12-07 21:14:00 | Leonard Cohen  | Thanks for the Dance            |           |
| 155479 | 2023-12-07 21:17:00 | The Vaccines   | Your Love Is My Favourite Band  |           |

|        | msPlayed |
|--------|----------|
| 94594  | 139.0    |
| 94595  | 487.0    |
| 94596  | 417.0    |
| 94597  | 2205.0   |
| 94598  | 278.0    |
| ...    | ...      |
| 155475 | 38298.0  |
| 155476 | 13448.0  |
| 155477 | 23757.0  |
| 155478 | 9317.0   |
| 155479 | 14661.0  |

[156539 rows x 4 columns]

## 2.4 Convert from milliseconds to seconds

From `msPlayed` , create a new column `secPlayed` with the data converted from milliseconds to seconds. Then delete the column `msPlayed` .

```
In [13]: df['secPlayed'] = df['msPlayed'] / 1000

df.drop(columns=['msPlayed'], inplace=True)

print(df.head(10))
```

|       | endTime             | artistName     | trackName \                  |
|-------|---------------------|----------------|------------------------------|
| 94594 | 2023-01-01 01:17:00 | Ariana Grande  | 7 rings                      |
| 94595 | 2023-01-01 01:17:00 | Ariana Grande  | 7 rings                      |
| 94596 | 2023-01-01 01:17:00 | Ariana Grande  | positions                    |
| 94597 | 2023-01-01 01:17:00 | Peach Pit      | Being so Normal              |
| 94598 | 2023-01-01 01:17:00 | Kelly Clarkson | Santa, Can't You Hear Me     |
| 94600 | 2023-01-01 01:17:00 | Ariana Grande  | Santa Baby                   |
| 94601 | 2023-01-01 01:17:00 | Ariana Grande  | Right There (feat. Big Sean) |
| 94599 | 2023-01-01 01:17:00 | Kelly Clarkson | Santa, Can't You Hear Me     |
| 94606 | 2023-01-01 01:19:00 | Regina Spektor | Us - 2005 Remaster           |
| 94605 | 2023-01-01 01:19:00 | Lorde          | Solar Power                  |

|       | secPlayed |
|-------|-----------|
| 94594 | 0.139     |
| 94595 | 0.487     |
| 94596 | 0.417     |
| 94597 | 2.205     |
| 94598 | 0.278     |
| 94600 | 12.293    |
| 94601 | 22.929    |
| 94599 | 0.325     |
| 94606 | 16.670    |
| 94605 | 0.464     |

## 2.5 Finding top 10 favorite artists

Find the top ten artists with the highest total play time (in seconds). Plot your findings in a bar graph.

(hint: start by creating a new DataFrame with only `artistName` and your time column. To proceed, you will also likely need the `groupby` command from Pandas.)

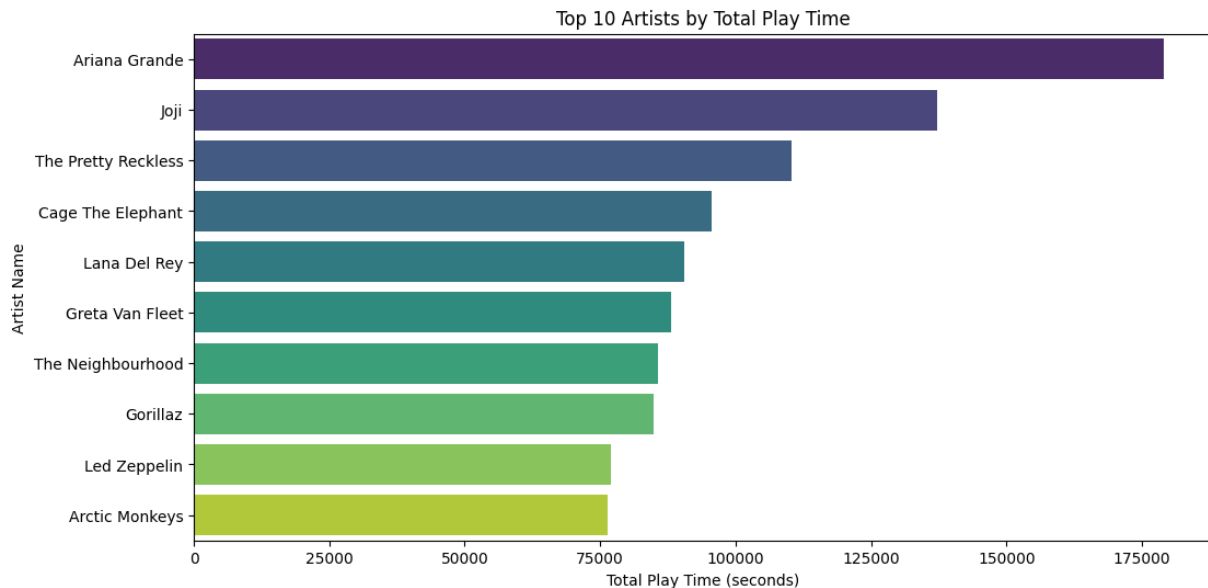
```
In [14]: # Group by artistName and sum the secPlayed for each artist
top_artists = df.groupby('artistName')['secPlayed'].sum().nlargest(10).reset_index()

# Plot the results
plt.figure(figsize=(12, 6))
sns.barplot(x='secPlayed', y='artistName', data=top_artists, palette='viridis')
plt.xlabel('Total Play Time (seconds)')
plt.ylabel('Artist Name')
plt.title('Top 10 Artists by Total Play Time')
plt.show()
print(df)
```

/tmp/ipykernel\_79394/3135675114.py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x='secPlayed', y='artistName', data=top_artists, palette='viridis')
```



|        | endTime             | artistName     | trackName                       |
|--------|---------------------|----------------|---------------------------------|
| \      |                     |                |                                 |
| 94594  | 2023-01-01 01:17:00 | Ariana Grande  | 7 rings                         |
| 94595  | 2023-01-01 01:17:00 | Ariana Grande  | 7 rings                         |
| 94596  | 2023-01-01 01:17:00 | Ariana Grande  | positions                       |
| 94597  | 2023-01-01 01:17:00 | Peach Pit      | Being so Normal                 |
| 94598  | 2023-01-01 01:17:00 | Kelly Clarkson | Santa, Can't You Hear Me        |
| ...    | ...                 | ...            | ...                             |
| 155475 | 2023-12-07 21:13:00 | Lana Del Rey   | Art Deco                        |
| 155476 | 2023-12-07 21:13:00 | Ariana Grande  | off the table (with The Weeknd) |
| 155477 | 2023-12-07 21:14:00 | Ariana Grande  | my hair                         |
| 155478 | 2023-12-07 21:14:00 | Leonard Cohen  | Thanks for the Dance            |
| 155479 | 2023-12-07 21:17:00 | The Vaccines   | Your Love Is My Favourite Band  |

|        | secPlayed |
|--------|-----------|
| 94594  | 0.139     |
| 94595  | 0.487     |
| 94596  | 0.417     |
| 94597  | 2.205     |
| 94598  | 0.278     |
| ...    | ...       |
| 155475 | 38.298    |
| 155476 | 13.448    |
| 155477 | 23.757    |
| 155478 | 9.317     |
| 155479 | 14.661    |

[156539 rows x 4 columns]

## 2.6 Finding top 10 favorite songs

Find the top ten songs with the highest play time. Create a graph visualizing the results.

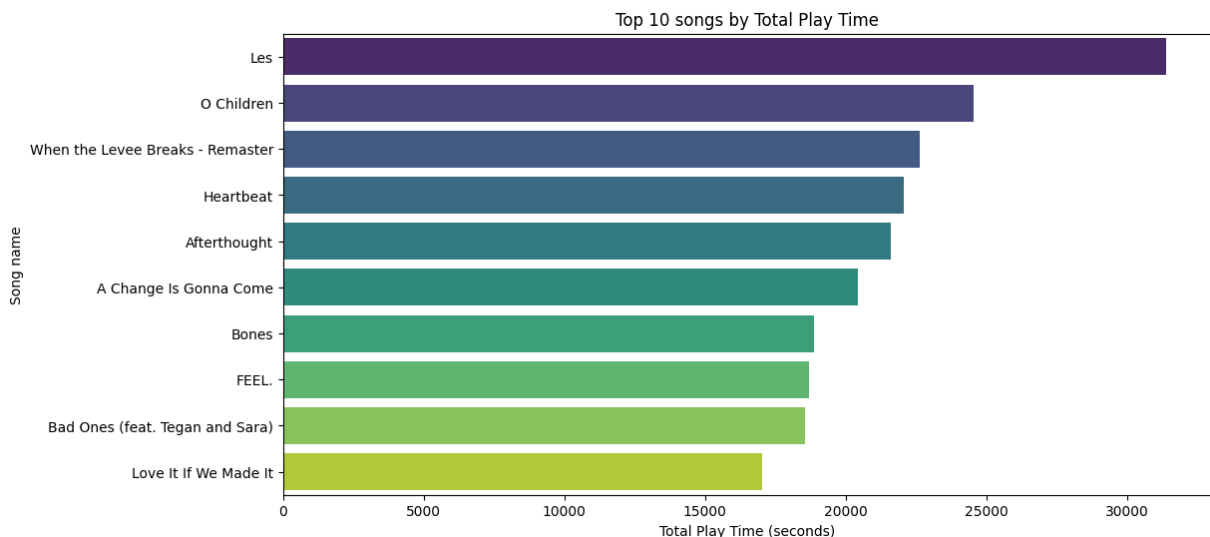
```
In [15]: # Group by songName and sum the secPlayed for each artist
top_songs = df.groupby('trackName')['secPlayed'].sum().nlargest(10).reset_in
```

```
# Plot the results
plt.figure(figsize=(12, 6))
sns.barplot(x='secPlayed', y='trackName', data=top_songs, palette='viridis')
plt.xlabel('Total Play Time (seconds)')
plt.ylabel('Song name')
plt.title('Top 10 songs by Total Play Time')
plt.show()
```

/tmp/ipykernel\_79394/1067714326.py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x='secPlayed', y='trackName', data=top_songs, palette='viridis')
```



## Part 3: Further analysis

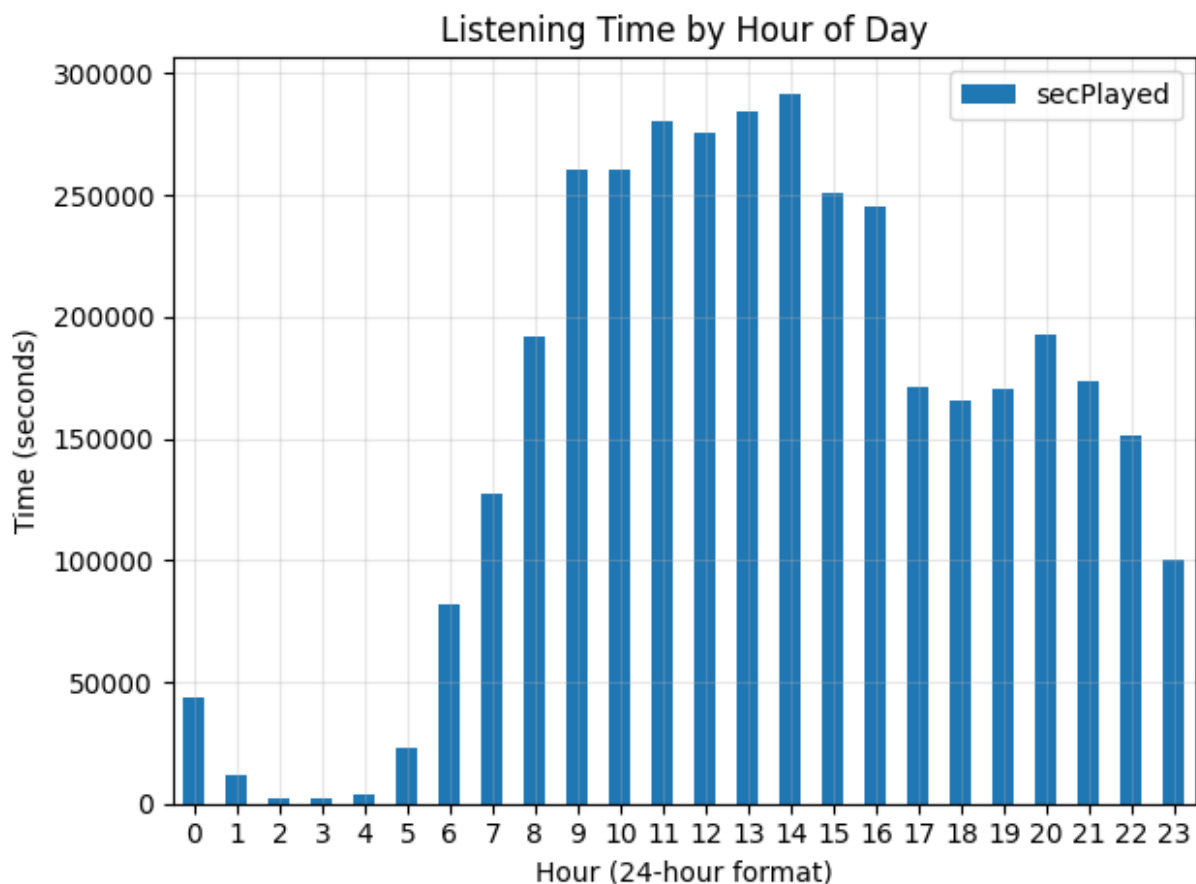
### 3.0 Average listening time by hour

Generate a plot that displays the average amount of time that music is played for each hour of the day.

```
In [16]: df["hour"] = df["endTime"].dt.hour
hourly_avg = df.groupby('hour')['secPlayed'].sum().reset_index()

plt.figure(figsize=(12, 6))
hourly_avg.plot(x='hour', y='secPlayed', kind='bar')
plt.title('Listening Time by Hour of Day')
plt.xlabel('Hour (24-hour format)')
plt.ylabel('Time (seconds)')
plt.grid(True, alpha=0.3)
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

<Figure size 1200x600 with 0 Axes>



### 3.1 Morning music and evening music

I think many people find that some types of music are more suitable for morning listening and some music is more suitable for evening listening. Create a plot that compares the play time of the artists *Leonard Cohen* and *Rage Against the Machine* on an hour-by-hour basis. See if there are any differences.

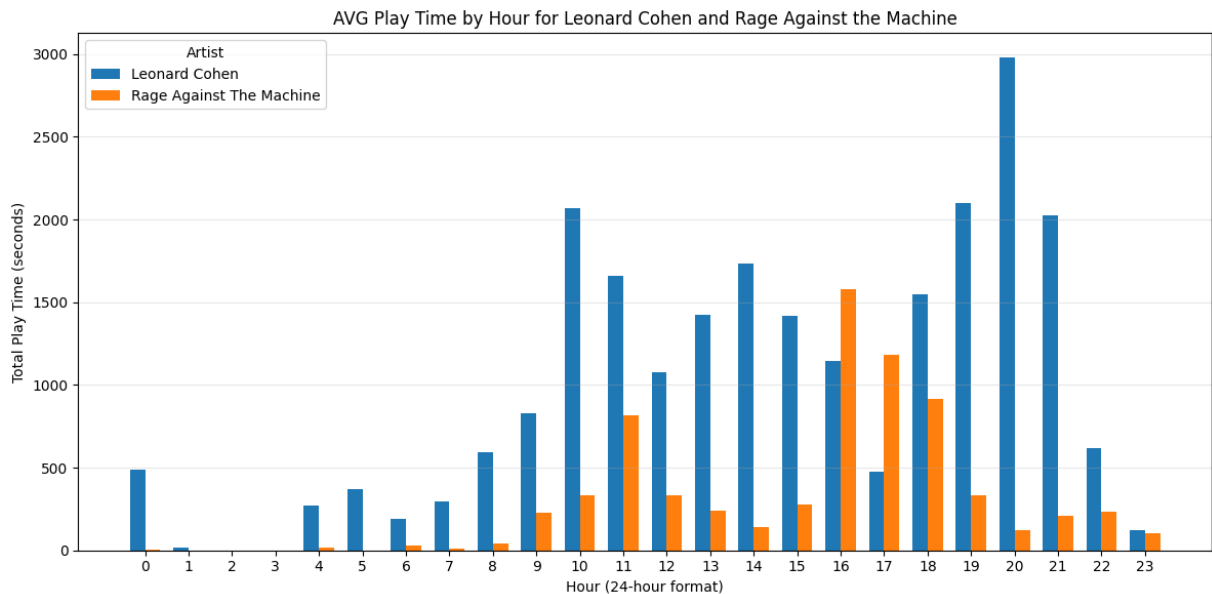
```
In [17]: df_artists = df[df["artistName"].isin(["Leonard Cohen", "Rage Against The Ma

hourly_playtime = df_artists.groupby(['hour', 'artistName'])['secPlayed'].su

plt.figure(figsize=(12, 6))
bar_width = 0.35
x = hourly_playtime.index

plt.bar(x - bar_width/2, hourly_playtime['Leonard Cohen'], bar_width, label=
plt.bar(x + bar_width/2, hourly_playtime['Rage Against The Machine'], bar_wi

plt.title('AVG Play Time by Hour for Leonard Cohen and Rage Against the Mach
plt.xlabel('Hour (24-hour format)')
plt.ylabel('Total Play Time (seconds)')
plt.xticks(ticks=range(0,24),rotation=0)
plt.legend(title='Artist')
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```



### 3.2 Analysing skipped songs

Determining whether a song was skipped or listened to can be challenging. For this analysis, we'll simplify by defining a skipped song as any track played for less than 30 seconds. Conversely, a song played for 30 seconds or more is considered listened to. Add a column to your DataFrame to reflect this criteria: set the value to 1 if the song was played for less than 30 seconds (indicating a skipped song), and 0 if it was played for 30 seconds or longer.

```
In [18]: df["Listened"] = (df["secPlayed"] < 30).astype(int)
df.head()
```

```
Out[18]:
```

|              | endTime                | artistName        | trackName                   | secPlayed | hour | Listened |
|--------------|------------------------|-------------------|-----------------------------|-----------|------|----------|
| <b>94594</b> | 2023-01-01<br>01:17:00 | Ariana<br>Grande  | 7 rings                     | 0.139     | 1    | 1        |
| <b>94595</b> | 2023-01-01<br>01:17:00 | Ariana<br>Grande  | 7 rings                     | 0.487     | 1    | 1        |
| <b>94596</b> | 2023-01-01<br>01:17:00 | Ariana<br>Grande  | positions                   | 0.417     | 1    | 1        |
| <b>94597</b> | 2023-01-01<br>01:17:00 | Peach Pit         | Being so Normal             | 2.205     | 1    | 1        |
| <b>94598</b> | 2023-01-01<br>01:17:00 | Kelly<br>Clarkson | Santa, Can't You<br>Hear Me | 0.278     | 1    | 1        |

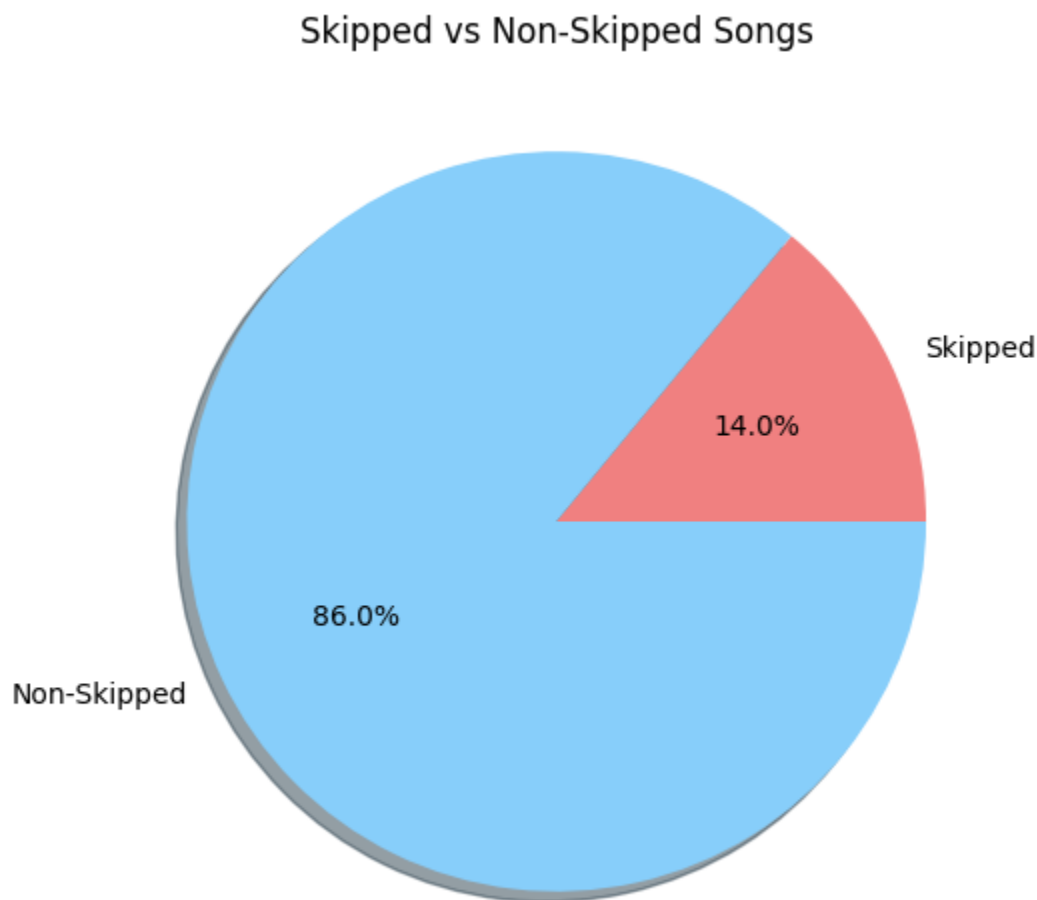
### 3.3 Plotting skipped songs

Create a pie-chart that compares amount of skipped songs to amount of non-skipped songs.



```
In [19]: skipped_count = len(df[df['Listened'] == 0])
non_skipped_count = len(df[df['Listened'] == 1])

plt.figure(figsize=(12, 6))
plt.pie([skipped_count, non_skipped_count], labels=['Skipped', 'Non-Skipped'],
        autopct='%1.1f%%', shadow=True)
plt.title('Skipped vs Non-Skipped Songs')
plt.show()
```



### 3.4 Artists by percentage of songs skipped

For each artist in the dataset, calculate which percentage of their songs was skipped. Store this information in a new DataFrame called `df_skipped`. Store the percentage of skipped songs in a new column named `SkipRate`

**Example:** If an artist has **100** songs in your dataset and **25** of these were skipped, the percentage of skipped songs for this artist would be  $\frac{25}{100} = 25\%$

```
In [20]: df_skipped = (df.groupby('artistName')['Listened'].sum() / ( df.groupby('art
df_skipped_given = df.groupby('artistName')['Listened'].mean().reset_index()
df_skipped_given.columns = ['artistName', 'skipRate']
```

```
df_skipped.head(10)
```

```
Out[20]:
```

|   | artistName           | skipRate |
|---|----------------------|----------|
| 0 | 10cc                 | 0.678571 |
| 1 | 2Pac                 | 0.861598 |
| 2 | 3 Doors Down         | 0.500000 |
| 3 | 4 Non Blondes        | 0.721311 |
| 4 | 50 Cent              | 0.678571 |
| 5 | 88rising             | 0.700375 |
| 6 | A Day To Remember    | 0.684211 |
| 7 | A Problem Squared    | 1.000000 |
| 8 | ARIZONA              | 0.800000 |
| 9 | A Tribe Called Quest | 0.792271 |

### 3.5 Comparing artists by skip-rate

Find the three top artists with the lowest skip-rate and the three with the highest. Print their names, along with their skip-rate.

```
In [21]: lowest_skip_rate = df_skipped.nsmallest(3, 'skipRate')
highest_skip_rate = df_skipped.nlargest(3, 'skipRate')

print("Three Artists with Lowest Skip-Rate:")
print(lowest_skip_rate[['artistName', 'skipRate']])

print("\nThree Artists with Highest Skip-Rate:")
print(highest_skip_rate[['artistName', 'skipRate']])
```

Three Artists with Lowest Skip-Rate:

|     | artistName    | skipRate |
|-----|---------------|----------|
| 305 | Gloria Gaynor | 0.000000 |
| 645 | Roc Boyz      | 0.111111 |
| 437 | LACES         | 0.142857 |

Three Artists with Highest Skip-Rate:

|    | artistName        | skipRate |
|----|-------------------|----------|
| 7  | A Problem Squared | 1.0      |
| 16 | Acid Ghost        | 1.0      |
| 25 | Albert Hammond Jr | 1.0      |

## Part 4: God Is a Data Scientist - The Ariana Deep-Dive

## 4.0 Ariana-DataFrame:

Create a new DataFrame called `df_ariana`, containing only rows with music by Ariana Grande.

```
In [22]: df_ariana = df[df["artistName"] == "Ariana Grande"]
df_ariana.head()
```

```
Out[22]:
```

|       | endTime                | artistName       | trackName                       | secPlayed | hour | Listened |
|-------|------------------------|------------------|---------------------------------|-----------|------|----------|
| 94594 | 2023-01-01<br>01:17:00 | Ariana<br>Grande | 7 rings                         | 0.139     | 1    | 1        |
| 94595 | 2023-01-01<br>01:17:00 | Ariana<br>Grande | 7 rings                         | 0.487     | 1    | 1        |
| 94596 | 2023-01-01<br>01:17:00 | Ariana<br>Grande | positions                       | 0.417     | 1    | 1        |
| 94600 | 2023-01-01<br>01:17:00 | Ariana<br>Grande | Santa Baby                      | 12.293    | 1    | 1        |
| 94601 | 2023-01-01<br>01:17:00 | Ariana<br>Grande | Right There (feat.<br>Big Sean) | 22.929    | 1    | 1        |

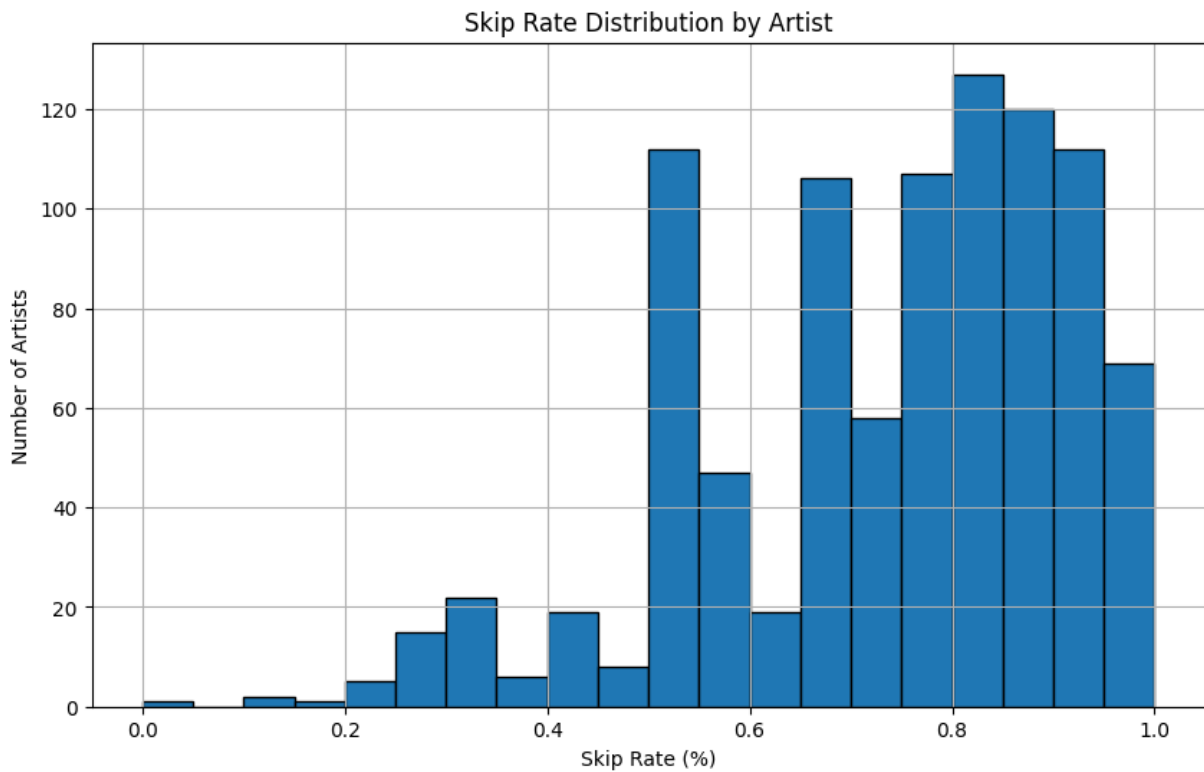
## 4.1 Average skip rate

Create a histogram of the distribution of the skip-rate values of the different artists in your DataFrame `df_skipped`, with skip rates on one axis and number of artists on the other.

Then, retrieve the skip rate for Ariana Grande from your DataFrame `df_skipped`. Run the code in the cell below. Where on this distribution does Ariana Grande fall? Do I skip her songs more than average, or less?

```
In [23]: plt.figure(figsize=(10, 6))
plt.hist(df_skipped['skipRate'], bins=20, edgecolor='black')
plt.title('Skip Rate Distribution by Artist')
plt.xlabel('Skip Rate (%)')
plt.ylabel('Number of Artists')
plt.grid(True)
plt.show()

df_ariana_skip_rate = df_skipped[df_skipped['artistName'] == "Ariana Grande"]
print(df_ariana_skip_rate)
```



```
artistName  skipRate
50 Ariana Grande  0.995294
```

## Part 4: Questions

Q1: Did I skip a lot of Ariana Grande's songs, or did I not, compared to the rest of the dataset?

A1: Yes, her you skipped a lot of ariana grande

Q2: What might be some possible reasons for Ariana Grande to be my nr.1 artist?

A2: Might have played many minutes of her songs, but still skipped in less then 30 seconds.